서론

이번 프로젝트에서는 수업에서 배운 핵심 개념들인 벡터 데이터베이스, ANN(Approximate Nearest Neighbor)검색, 하이브리드 검색(메타데이터 필터링), Upsert 와 멱등성을 직접 적용하여, 단순히 파일에 저장된 임베딩이 아닌 확장 가능하고 운영 가능한(production-ready) AI 애플리케이션을 구현을 목표로 진행하였다. 수업에서 배운 "파일 기반 임베딩 시스템은 데이터가 늘어날수록 무너진다", "벡터 DB 는 검색 알고리즘과 영속성, 메타데이터 관리를 한 번에 해결한다" 는 통찰을 직접 코드로 체험하는 것이 이 프로젝트의 핵심 목표이며 또한, ChromaDB 의 "텍스트를 주면 벡터로 만들어주는" 접근과 Pinecone 의 "벡터를 주면 저장/검색해주는" 접근의 차이를 실제 코드로 비교 경험하였다.

프로젝트 배경 및 동기

현대의 검색 시스템은 단순 키워드 매칭에서 벗어나 사용자의 복잡한 의도와 문맥을 파악하는 방향으로 진화하고 있다. 본 프로젝트는 이러한 흐름에 맞춰 사용자의 모호한 질의에도 최적의 결과를 도출하는 시스템을 제안하기 위해 **지능형 검색의 필요성**을 느끼고 프로젝트를 진행하였다. **또한, 도서 도메인의 특성**으로 도서는 줄거리, 주제, 문체 등 텍스트 데이터가 풍부하여 벡터 임베딩을 통한 시맨틱 검색 효율이 매우 높은 분야이기 때문에 적합하다고 판단하였다.

기존 시스템(파일 기반)의 기술적 부채 및 한계

계산 복잡도의 문제이다. 데이터셋이 N 개일 때, 유사도 계산은 O(N x D) (D 는 차원 수)의 연산량을 가진다. 이전 과제의 50 개 수준에서는 체감되지 않았으나, 200 개 이상으로 확장 시 응답 지연이 눈에 띄게 발생함을 확인하였다.

동적 업데이트의 어려움이 존재한다. JSON/Pickle 기반 시스템은 데이터 하나를 수정하더라도 파일 전체를 다시 쓰거나 메모리에 전체를 로드해야 하는 비효율성이 존재한다.

메타데이터의 파편화 문제이다. 텍스트 벡터와 메타데이터(평점, 장르 등)가 분리되어 관리되므로, 복합적인 '하이브리드 쿼리'를 구현하기 위해선 개발자가 수동으로 필터링 로직을 작성해야 하는 번거로움이 있다.

배경 지식

임베딩(Embedding)의 개념과 의미적 거리

임베딩은 컴퓨터가 텍스트의 맥락과 의미를 이해할 수 있도록 문장이나 단어를 고차원 공간상의 수치 벡터로 변환하는 기술이다. 단순히 단어를 숫자로 치환하는 것을 넘어, 언어 모델(예: text-embedding-3-small)을 통해 텍스트 데이터의 특징을 추출하여 수천

차원의 공간에 좌표로 매핑한다. 이 과정의 핵심은 **'의미적 유사성'을 '기하학적 거리'로 변환**하는 데 있다. 예를 들어, "추리 소설"과 "탐정 이야기"는 단어 구성은 다르지만 임베딩 공간상에서는 매우 가까운 거리(예: 코사인 유사도 점수 기반)에 위치하게 된다. 이를 통해 시스템은 키워드가 일치하지 않아도 사용자의 의도를 파악한 시맨틱 검색을 수행할 수 있습니다.

ANN(Approximate Nearest Neighbor)

데이터 규모가 수만 건 이상으로 커지면 모든 벡터 간의 거리를 계산하는 Brute-force 방식은 실시간 서비스에 부적합하다. 이를 해결하기 위해 ChromaDB 등 현대적인 벡터 DB 는 HNSW(Hierarchical Navigable Small World)와 같은 그래프 기반 인덱싱 기법을 사용한다. 이 알고리즘에는 **HNSW 의 원리**가 사용된다. 데이터를 여러 층(Layer)의 그래프 구조로 관리한다. 상위 층은 데이터가 듬성듬성 배치되어 있어 '스킵 리스트'처럼 넓은 범위를 빠르게 건너뛸 수 있게 하고, 하위 층으로 내려갈수록 촘촘한 노드 사이에서 정밀하게 가장 가까운 이웃을 찾아낸다. 이는 모든 점을 검사하는 대신, 상위 층에서 대략적인 방향을 정하고 하위 층에서 미세 조정하는 효율적인 탐색을 가능하게 한다.

검색의 트레이드오프(Trade-off)

ANN 알고리즘의 핵심 철학은 '검색 속도와 정확도 사이의 타협'이다. 100% 완벽하게 가장 가까운 점을 찾기 위해 수 초를 기다리는 대신, 99%의 확률로 가장 유사한 결과를 밀리초(ms) 단위로 제공하는 것이다. 이러한 미세한 정확도 희생은 대규모 시스템에서 검색 속도를 수백 배 이상 향상시키며, 사용자 경험(UX) 측면에서 훨씬 더 큰 가치를 제공한다.

하이브리드 검색의 메커니즘

Semantic + Filter: 단순 벡터 유사도는 "무거운 느낌의 소설"은 잘 찾지만 "500 페이지 이하"라는 물리적 제약은 이해하지 못한다. 이를 해결하기 위해 벡터 검색 결과에 불리언 필터(Boolean Filter)를 적용하는 하이브리드 방식의 작동 원리를 설명한다.

Upsert 와 멱등성 (Idempotency)의 중요성

운영 안정성으로, API 통신 중 네트워크 오류로 인해 데이터가 중복 전송되는 시나리오를 가정한다. Upsert 가 ID 를 기준으로 기존 데이터를 덮어씀으로써 DB 의 정합성을 유지하고 시스템 부하를 줄이는 과정을 서술한다.

시스템 설계 및 구현

아키텍처 및 데이터 흐름

전체 구조: Streamlit(Front-end) ↔ OpenAI API(Embedding)
↔ ChromaDB(Vector Engine) ↔ Naver API(Data Source).

메타데이터 스키마 설계 근거:

- title: 사용자에게 결과를 식별 가능하게 함.
- genre: eq 연산자를 통한 카테고리 분류.
- rating: gte 연산자를 통한 품질 보증 필터.
- pages: lte 연산자를 통한 분량 제어 필터.

구현 상세 및 CRUD 로직

Custom Embedding Function: ChromaDB 내부에 OpenAI 임베딩 함수를 직접 등록하여 텍스트만 주면 자동으로 임베딩되는 과정을 코드와 함께 설명합니다.

Update vs Upsert 차이 (핵심):

Update: ID 가 존재하지 않으면 검색 자체를 시도하지 않음
Upsert: 데이터 동기화 파이프라인에서 가장 효율적인 방식

결과 분석 및 고찰

정량적 비교 분석

데이터 규모	파일 기반 검색 속도(s)	벡터 DB(Chroma) 속도(s)
100 건	0.005	0.004
500 건	0.025	0.005
1,000 건(예측)	0.055	0.006

데이터가 늘어날수록 벡터 DB 의 인덱싱 효율이 기하급수적으로 좋아진다.

시맨틱 접근의 한계 및 개선 방향

시스템의 한계: 짧은 데이터셋의 정보 부족과 유사도 왜곡

본 프로젝트에서 구축한 시스템은 데이터의 양과 질에 따라 검색 품질이 크게 좌우되는 특성을 보였으며 특히 다음과 같은 한계점이 관찰되었다.

정보 부족으로 인한 의미적 편향이 일어난다. 네이버 도서 API 에서 제공하는 줄거리(description)가 한두 문장으로 매우 짧은 경우, 임베딩 모델이 해당 도서의 전체적인 맥락을 파악하기에 충분한 특징(Feature)을 추출하지 못한다. 이로 인해 검색어와 줄거리 사이의 심층적인 의미 연결이 약해지고, 결과적으로 검색 품질이 저하되는 현상이 발생한다.

특정 키워드에 대한 과도한 가중치(Keyword Over-weighting)에 관한 문제로 시맨틱 검색은 문맥을 파악해야 함에도 불구하고, 데이터가 부족한 상황에서는 제목이나 줄거리에 포함된 특정 단어에 유사도 점수가 쏠리는 경향이 나타난다. 사용자가 "성장을 다룬 소설"을 검색했을 때, 실제 내용은 성장을 다루고 있지만 '성장'이라는 단어가 없는 도서보다, 제목에 단순히 '성장'이라는 단어가 포함된 도서가 거리 점수(Distance) 상위권을 차지하는 '키워드 매칭'으로의 퇴행 현상이 관찰되었으며, 고차원 벡터 공간에서 짧은 텍스트는 벡터의 방향성이 명확하게 정립되지 않아, 특정 핵심 단어의 벡터 성분이 전체 유사도를 지배하기 때문으로 보인다.

향후 개선 방향:

BM25 와 벡터 검색의 결합 (Reciprocal Rank Fusion)
단순 벡터 검색의 한계를 보완하기 위해, 전통적인 키워드 검색 알고리즘인 **BM25** 와 벡터 기반 시맨틱 검색을 결합하는 **RRF(Reciprocal Rank Fusion)** 기법을 도입할 수 있다. 이는 키워드의 정확성과 문맥의 유연함을 동시에 확보하여 검색 결과의 왜곡을 방지한다.

재순위화(Re-ranking) 단계 도입
ChromaDB 에서 1 차적으로 유사한 후보군(예: 상위 50 개)을 빠르게 추출한 뒤, 문장 간의 관계를 더 정밀하게 파악하는 **Cross-Encoder** 모델을 활용하여 순위를 다시 매기는 과정을 추가한다. 이 과정은 연산 비용은 높지만, 짧은 문장에서도 높은 수준의 의미적 일치도를 보장한다.

청킹(Chunking) 및 데이터 증강(Data Augmentation)
도서의 줄거리뿐만 아니라 목차, 서평, 저자 소개 등 추가적인 메타데이터를 결합하여

임베딩 대상 텍스트의 길이를 충분히 확보함으로써 벡터의 의미적 밀도를 높이는 전략이 필요하다.

결론

첫째, **운영 효율성** 측면에서 `PersistentClient`를 활용해 데이터의 영속성을 확보하였으며, `Upsert` 로직을 구현하여 데이터의 중복을 방지하고 시스템의 멱등성을 증명하는 과정을 추가하여 실제 운영 환경에서 발생할 수 있는 네트워크 오류나 중복 요청에도 시스템이 일관된 상태를 유지할 수 있다.

둘째, **검색 고도화** 측면에서 단순 시맨틱 검색의 모호함을 메타데이터 필터링(eq, qte, lte)으로 보완하는 하이브리드 검색을 구현하였다. 이를 통해 사용자는 자신의 주관적인 느낌(질의어)과 객관적인 제약 조건(장르, 평점, 분량)을 동시에 만족하는 고도화된 추천 경험을 누릴 수 있게 되었다.

참고문헌

1. ChromaDB Documentation (https://docs.trychroma.com)
2. OpenAI API Documentation (https://platform.openai.com/docs/guides/embeddings)